# Observability

# 1. Overview

Monitoring tells you **what is wrong**.
Observability helps you understand **why it is wrong**.

> Observability is the ability to infer the internal state of a system from its external outputs.

Those outputs are mainly:

- Metrics
- Logs
- Traces

Without observability, your system is a **black box**.

# 2. Why Observability Matters

Modern distributed systems include:

- Microservices
- Queues
- Caches
- Databases
- Third-party APIs
- Multiple regions

When latency spikes or failures happen, observability helps answer:

- Which service failed?
- Where did latency increase?
- Which release caused it?

- Which users are impacted?
- Is this infra, code, or dependency issue?

# 3. Monitoring vs Observability

| Concept | Meaning |
|---|---|
| Monitoring | Watching known metrics & alerts |
| Observability | Investigating unknown issues deeply |

**Example:**

Monitoring says:

> P99 latency increased.

Observability says:

> Latency increased only for `/checkout`, only in EU region, after version v142, due to DB connection pool exhaustion.

# 4. Three Pillars of Observability

# A. Metrics

Aggregated numeric values over time.

Examples:

- RPS
- Error rate
- P99 latency
- CPU
- Cache hit rate

# B. Logs

Detailed event records.

```
None
request_id=abc123

user_id=52

endpoint=/payment

error=timeout
```

# C. Traces

Follow one request across systems.

```
None
Client

 ↓

API Gateway

 ↓

Auth Service

 ↓

Order Service

 ↓

Payment Service
```

# 5. Golden Signals (Google SRE)

Excellent interview topic.

## 1. Latency

How long requests take.

Track:

- P50
- P95
- P99

Alert if SLA breached.

## 2. Traffic

Demand on system.

Examples:

- Requests/sec
- Messages/sec
- Queries/sec

## 3. Errors

Failed requests.

Examples:

- 4xx
- 5xx
- Timeouts

- Retries exhausted

### 4. Saturation

How close resources are to limits.

Examples:

- CPU 90%
- Memory full
- Queue lag
- DB connections maxed

# 6. Why P99 Matters

Average latency hides pain.

Example:

- Avg = 120ms
- P99 = 4 sec

Meaning top 1% users suffer badly.

Top candidates always mention percentiles.

# 7. Dashboards

Dashboards summarize system state.

### Typical Dashboard

None

```
Traffic: 12k RPS

P99: 280ms

Errors: 0.2%

CPU: 61%

DB Lag: 0

Cache Hit: 94%
```

# 8. Alerts

Alerts should be actionable.

## Good Alerts

- P99 > 1 sec for 5 min
- Error rate > 2%
- Queue lag > 50k
- Disk > 90%

## Bad Alerts

- CPU = 72%
- Single transient error
- Too noisy

# 9. Structured Logging Best Practices

Excellent interview point.

## Use JSON / Structured Logs

```json
{
 "timestamp":"2026-04-24T11:20:00Z",
 "level":"ERROR",
 "service":"checkout",
 "request_id":"abc123",
 "user_id":"52"
}
```

### Benefits

- Searchable
- Machine-readable
- Easy dashboards

# 10. Correlation IDs / Request IDs

Critical in microservices.

Same request ID flows across services.

```
X-Request-ID: abc123
```

Use it to trace user journey.

# 11. Time Consistency

All logs should use:

- UTC timezone
- ISO format

Avoid mixed timezones.

# 12. Sensitive Data Rules

Never log:

- Passwords
- Tokens
- Credit cards
- PII unnecessarily

Mask data where needed.

# 13. Business Observability

Beyond servers, observe product health.

Examples:

- Orders/minute
- Checkout success rate
- Signup conversion
- Search click-through rate

# 14. Silent Failures

Most dangerous failures:

Request returns 200 OK but wrong behavior.

Examples:

- Email not actually sent
- Payment marked success but not charged
- Cache returns stale data

Need audits + reconciliation jobs.

# 15. Data Integrity Audits

Run periodic jobs:

```
None
Orders DB count vs Payment DB count

Inventory vs Actual stock

Sent notifications vs Delivered
```

Alert on mismatch.

# 16. Observability for Cache Example

Track:

- Hits
- Misses
- Evictions
- Latency

Without this, cache tuning is blind.

# 17. Common Tools

- Prometheus
- Grafana
- Datadog
- New Relic
- OpenTelemetry
- ELK Stack
- Jaeger / Zipkin

# 18. Interview Script

Say:

> “I’d add observability using metrics, structured logs, distributed tracing, dashboards, and alerts. I’d track golden signals plus business KPIs, and use request IDs for debugging cross-service flows.”

Strong answer.

# 19. Final Summary

- Monitoring = detect issues
- Observability = understand issues
- Pillars = metrics, logs, traces
- Golden signals = latency, traffic, errors, saturation
- Use dashboards + alerts
- Include business metrics & audits

# 20. Memorable Line

> Monitoring tells you the fire exists. Observability helps find where it started.